

```

input1 = input("What is your username: ")
while input2 != password:
    input2 = input("What is your password: ")
print("Welcome back to your system!")

```

If you would like the program to run continuously, just add a `while 1 == 1:` loop around the whole thing. You will have to indent the rest of the program when you add this at the top of the code, but don't worry, you don't have to do it manually for each line! Just highlight everything you want to indent and click on "Indent" under "Format" in the top bar of the python window.

Another way of doing this could be:

```

name = input('Set name: ')
password = input('Set password: ')
while 1 == 1:
    nameguess=""
    passwordguess=""
    key=""
    while (nameguess != name) or (passwordguess != password):
        nameguess = input('Name? ')
        passwordguess = input('Password? ')
    print("Welcome,", name, ". Type lock to lock.")
    while key != "lock":
        key = input("")

```

Notice the `or` in `while (nameguess != name) or (passwordguess != password)`, which we haven't yet introduced. You can probably figure out how it works.

ca:Python 3 per a no programadors/Comptar fins a 10¹

¹ <https://ca.wikibooks.org/wiki/Python%20per%20a%20no%20programadors%2FComptar%20fins%20a%2010>

6 Decisions

6.0.1 If statement

As always, I believe I should start each chapter with a warm-up typing exercise, so here is a short program to compute the absolute value of an integer:

```
n = int(input("Number? "))
if n < 0:
    print("The absolute value of", n, "is", -n)
else:
    print("The absolute value of", n, "is", n)
```

Here is the output from the two times that I ran this program:

```
Number? -34
The absolute value of -34 is 34
```

```
Number? 1
The absolute value of 1 is 1
```

So what does the computer do when it sees this piece of code? First it prompts the user for a number with the statement `n = int(input("Number? "))`. Next it reads the line `if n < 0:`. If `n` is less than zero Python runs the line `print("The absolute value of", n, "is", -n)`. Otherwise it runs the line `print("The absolute value of", n, "is", n)`.

More formally Python looks at whether the *expression* `n < 0` is true or false. An `if` statement is followed by an indented *block* of statements that are run when the expression is true. Optionally after the `if` statement is an `else` statement and another indented *block* of statements. This second block of statements is run if the expression is false.

There are a number of different tests that an expression can have. Here is a table of all of them:

operator	function
<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to
==	equal
!=	not equal

Another feature of the `if` command is the `elif` statement. It stands for else if and means if the original `if` statement is false but the `elif` part is true, then do the `elif` part. And

if neither the `if` or `elif` expressions are true, then do what's in the `else` block. Here's an example:

```
a = 0
while a < 10:
    a = a + 1
    if a > 5:
        print(a, ">", 5)
    elif a <= 3:
        print(a, "<=", 3)
    else:
        print("Neither test was true")
```

and the output:

```
1 <= 3
2 <= 3
3 <= 3
Neither test was true
Neither test was true
6 > 5
7 > 5
8 > 5
9 > 5
10 > 5
```

Notice how the `elif a <= 3` is only tested when the `if` statement fails to be true. There can be more than one `elif` expression, allowing multiple tests to be done in a single `if` statement.

6.0.2 Examples

```
# This Program Demonstrates the use of the == operator
# using numbers
print(5 == 6)
# Using variables
x = 5
y = 8
print(x == y)
```

And the output

```
False
False
```

high_low.py

```
# Plays the guessing game higher or lower

# This should actually be something that is semi random like the
# last digits of the time or something else, but that will have to
# wait till a later chapter. (Extra Credit, modify it to be random
# after the Modules chapter)
number = 7
guess = -1

print("Guess the number!")
while guess != number:
```

```
guess = int(input("Is it... "))

if guess == number:
    print("Hooray! You guessed it right!")
elif guess < number:
    print("It's bigger...")
elif guess > number:
    print("It's not so big.")
```

Sample run:

```
Guess the number!
Is it... 2
It's bigger...
Is it... 5
It's bigger...
Is it... 10
It's not so big.
Is it... 7
Hooray! You guessed it right!
```

even.py

```
# Asks for a number.
# Prints if it is even or odd

number = float(input("Tell me a number: "))
if number % 2 == 0:
    print(int(number), "is even.")
elif number % 2 == 1:
    print(int(number), "is odd.")
else:
    print(number, "is very strange.")
```

Sample runs:

```
Tell me a number: 3
3 is odd.
```

```
Tell me a number: 2
2 is even.
```

```
Tell me a number: 3.4895
3.4895 is very strange.
```

average1.py

```
# keeps asking for numbers until 0 is entered.
# Prints the average value.

count = 0
sum = 0.0
number = 1 # set to something that will not exit the while loop immediately.

print("Enter 0 to exit the loop")

while number != 0:
```

```
number = float(input("Enter a number: "))
if number != 0:
    count = count + 1
    sum = sum + number
if number == 0:
    print("The average was:", sum / count)
```

Sample runs

Sample runs:

```
Enter 0 to exit the loop
Enter a number: 3
Enter a number: 5
Enter a number: 0
The average was: 4.0
```

```
Enter 0 to exit the loop
Enter a number: 1
Enter a number: 4
Enter a number: 3
Enter a number: 0
The average was: 2.66666666667
```

average2.py

```
# keeps asking for numbers until count numbers have been entered.
# Prints the average value.

#Notice that we use an integer to keep track of how many numbers,
# but floating point numbers for the input of each number
sum = 0.0

print("This program will take several numbers then average them")
count = int(input("How many numbers would you like to average: "))
current_count = 0

while current_count < count:
    current_count = current_count + 1
    print("Number", current_count)
    number = float(input("Enter a number: "))
    sum = sum + number

print("The average was:", sum / count)
```

Sample runs:

```
This program will take several numbers then average them
How many numbers would you like to average: 2
Number 1
Enter a number: 3
Number 2
Enter a number: 5
The average was: 4.0
```

```
This program will take several numbers then average them
How many numbers would you like to average: 3
Number 1
Enter a number: 1
Number 2
Enter a number: 4
Number 3
Enter a number: 3
The average was: 2.66666666667
```

6.0.3 Exercises

Write a program that asks the user their name, if they enter your name say "That is a nice name", if they enter "John Cleese" or "Michael Palin", tell them how you feel about them ;), otherwise tell them "You have a nice name."

Solution

```
name = input('Your name: ')
if name == 'Bryn':
    print('That is a nice name.')
elif name == 'John Cleese':
    print('... some funny text.')
elif name == 'Michael Palin':
    print('... some funny text.')
else:
    print('You have a nice name.')
```

Modify the higher or lower program from this section to keep track of how many times the user has entered the wrong number. If it is more than 3 times, print "That must have been complicated." at the end, otherwise print "Good job!"

Solution

```
number = 7
guess = -1
count = 0

print("Guess the number!")
while guess != number:
    guess = int(input("Is it... "))
    count = count + 1
    if guess == number:
        print("Hooray! You guessed it right!")
    elif guess < number:
        print("It's bigger...")
    elif guess > number:
        print("It's not so big.")

if count > 3:
    print("That must have been complicated.")
else:
    print("Good job!")
```

Write a program that asks for two numbers. If the sum of the numbers is greater than 100, print "That is a big number."

Solution

```
number1 = float(input('1st number: '))
number2 = float(input('2nd number: '))
if number1 + number2 > 100:
    print('That is a big number.')
```


7 Debugging

7.0.1 What is debugging?

"As soon as we started programming, we found to our surprise that it wasn't as easy to get programs right as we had thought. Debugging had to be discovered. I can remember the exact instant when I realized that a large part of my life from then on was going to be spent in finding mistakes in my own programs." — *Maurice Wilkes discovers debugging*, 1949

By now if you have been messing around with the programs you have probably found that sometimes the program does something you didn't want it to do. This is fairly common. Debugging is the process of figuring out what the computer is doing and then getting it to do what you want it to do. This can be tricky. I once spent nearly a week tracking down and fixing a bug that was caused by someone putting an `x` where a `y` should have been.

This chapter will be more abstract than previous chapters.

7.0.2 What should the program do?

The first thing to do (this sounds obvious) is to figure out what the program should be doing if it is running correctly. Come up with some test cases and see what happens. For example, let's say I have a program to compute the perimeter of a rectangle (the sum of the length of all the edges). I have the following test cases:

height	width	perimeter
3	4	14
2	3	10
4	4	16
2	2	8
5	1	12

I now run my program on all of the test cases and see if the program does what I expect it to do. If it doesn't then I need to find out what the computer is doing.

More commonly some of the test cases will work and some will not. If that is the case you should try and figure out what the working ones have in common. For example here is the output for a perimeter program (you get to see the code in a minute):

```
Height: 3
Width: 4
perimeter = 15
```

```
Height: 2
Width: 3
perimeter = 11
```

```
Height: 4
Width: 4
perimeter = 16
```

```
Height: 2
Width: 2
perimeter = 8
```

```
Height: 5
Width: 1
perimeter = 8
```

Notice that it didn't work for the first two inputs, it worked for the next two and it didn't work on the last one. Try and figure out what is in common with the working ones. Once you have some idea what the problem is finding the cause is easier. With your own programs you should try more test cases if you need them.

7.0.3 What does the program do?

The next thing to do is to look at the source code. One of the most important things to do while programming is reading source code. The primary way to do this is code walkthroughs.

A code walkthrough starts at the first line, and works its way down until the program is done. `while` loops and `if` statements mean that some lines may never be run and some lines are run many times. At each line you figure out what Python has done.

Lets start with the simple perimeter program. Don't type it in, you are going to read it, not run it. The source code is:

```
height = int(input("Height: "))
width = int(input("Width: "))
print("perimeter =", width + height + width + width)
```

Question: What is the first line Python runs?

Answer: The first line is always run first. In this case it is: `height = int(input("Height: "))`

What does that line do?

Prints `Height:`, waits for the user to type a string in, and then converts the string to an integer variable `height`.

What is the next line that runs?

In general, it is the next line down which is: `width = int(input("Width: "))`

What does that line do?

Prints Width:, waits for the user to type a number in, and puts what the user types in the variable width.

What is the next line that runs?

When the next line is not indented more or less than the current line, it is the line right afterwards, so it is: `print("perimeter = ", width + height + width + width)` (It may also run a function in the current line, but that's a future chapter.)

What does that line do?

First it prints `perimeter =`, then it prints the sum of the values contained within the variables, `width` and `height`, from `width + height + width + width`.

Does `width + height + width + width` calculate the perimeter properly?

Let's see, perimeter of a rectangle is the bottom (`width`) plus the left side (`height`) plus the top (`width`) plus the right side (huh?). The last item should be the right side's length, or the `height`.

Do you understand why some of the times the perimeter was calculated "correctly"?

It was calculated correctly when the `width` and the `height` were equal.

The next program we will do a code walkthrough for is a program that is supposed to print out 5 dots on the screen. However, this is what the program is outputting:

```
. . . .
```

And here is the program:

```
number = 5
while number > 1:
    print(".",end=" ")
    number = number - 1
print()
```

This program will be more complex to walkthrough since it now has indented portions (or control structures). Let us begin.

What is the first line to be run?

The first line of the file: `number = 5`

What does it do?

Puts the number 5 in the variable `number`.

What is the next line?

The next line is: `while number > 1:`

What does it do?

Well, `while` statements in general look at their expression, and if it is true they do the next indented block of code, otherwise they skip the next indented block of code.

So what does it do right now?

If `number > 1` is true then the next two lines will be run.

So is `number > 1`?

The last value put into `number` was 5 and `5 > 1` so yes.

So what is the next line?

Since the `while` was true the next line is: `print(".",end=" ")`

What does that line do?

Prints one dot and since the extra argument `end=" "` exists the next printed text will not be on a different screen line.

What is the next line?

`number = number - 1` since that is following line and there are no indent changes.

What does it do?

It calculates `number - 1`, which is the current value of `number` (or 5) subtracts 1 from it, and makes that the new value of `number`. So basically it changes `number`'s value from 5 to 4.

What is the next line?

Well, the indent level decreases so we have to look at what type of control structure it is. It is a `while` loop, so we have to go back to the `while` clause which is `while number > 1`:

What does it do?

It looks at the value of `number`, which is 4, and compares it to 1 and since `4 > 1` the `while` loop continues.

What is the next line?

Since the `while` loop was true, the next line is: `print(".",end=" ")`

What does it do?

It prints a second dot on the line, ending by a space.

What is the next line?

No indent change so it is: `number = number - 1`

And what does it do?

It takes the current value of `number` (4), subtracts 1 from it, which gives it 3 and then finally makes 3 the new value of `number`.

What is the next line?

Since there is an indent change caused by the end of the `while` loop, the next line is: `while number > 1`:

What does it do?

It compares the current value of number (3) to 1. $3 > 1$ so the while loop continues.

What is the next line?

Since the while loop condition was true the next line is: `print(".",end=" ")`

And it does what?

A third dot is printed on the line.

What is the next line?

It is: `number = number - 1`

What does it do?

It takes the current value of number (3) subtracts from it 1 and makes the 2 the new value of number.

What is the next line?

Back up to the start of the while loop: `while number > 1:`

What does it do?

It compares the current value of number (2) to 1. Since $2 > 1$ the while loop continues.

What is the next line?

Since the while loop is continuing: `print(".",end=" ")`

What does it do?

It discovers the meaning of life, the universe and everything. I'm joking. (I had to make sure you were awake.) The line prints a fourth dot on the screen.

What is the next line?

It's: `number = number - 1`

What does it do?

Takes the current value of number (2) subtracts 1 and makes 1 the new value of number.

What is the next line?

Back up to the while loop: `while number > 1:`

What does the line do?

It compares the current value of number (1) to 1. Since $1 > 1$ is false (one is not greater than one), the while loop exits.

What is the next line?

Since the while loop condition was false the next line is the line after the while loop exits, or: `print()`

What does that line do?

Makes the screen go to the next line.

Why doesn't the program print 5 dots?

The loop exits 1 dot too soon.

How can we fix that?

Make the loop exit 1 dot later.

And how do we do that?

There are several ways. One way would be to change the while loop to: `while number > 0`: Another way would be to change the conditional to: `number >= 1` There are a couple others.

7.0.4 How do I fix my program?

You need to figure out what the program is doing. You need to figure out what the program should do. Figure out what the difference between the two is. Debugging is a skill that has to be practiced to be learned. If you can't figure it out after an hour, take a break, talk to someone about the problem or contemplate the lint in your navel. Come back in a while and you will probably have new ideas about the problem. Good luck.

8 Defining Functions

8.0.1 Creating Functions

To start off this chapter I am going to give you an example of what you could do but shouldn't (so don't type it in):

```
a = 23
b = -23

if a < 0:
    a = -a
if b < 0:
    b = -b
if a == b:
    print("The absolute values of", a, "and", b, "are equal.")
else:
    print("The absolute values of", a, "and", b, "are different.")
```

with the output being:

```
The absolute values of 23 and 23 are equal.
```

The program seems a little repetitive. Programmers hate to repeat things – that's what computers are for, after all! (Note also that finding the absolute value changed the value of the variable, which is why it is printing out 23, and not -23 in the output.) Fortunately Python allows you to create functions to remove duplication. Here is the rewritten example:

```
a = 23
b = -23

def absolute_value(n):
    if n < 0:
        n = -n
    return n

if absolute_value(a) == absolute_value(b):
    print("The absolute values of", a, "and", b, "are equal.")
else:
    print("The absolute values of", a, "and", b, "are different.")
```

with the output being:

```
The absolute values of 23 and -23 are equal.
```

The key feature of this program is the **def** statement. **def** (short for define) starts a function definition. **def** is followed by the name of the function **absolute_value**. Next comes a '(' followed by the parameter **n** (**n** is passed from the program into the function when the

function is called). The statements after the ':' are executed when the function is used. The statements continue until either the indented statements end or a **return** is encountered. The **return** statement returns a value back to the place where the function was called. We already have encountered a function in our very first program, the **print** function. Now we can make new functions.

Notice how the values of **a** and **b** are not changed. Functions can be used to repeat tasks that don't return values. Here are some examples:

```
def hello():
    print("Hello")

def area(width, height):
    return width * height

def print_welcome(name):
    print("Welcome", name)

hello()
hello()

print_welcome("Fred")
w = 4
h = 5
print("width =", w, " height =", h, " area =", area(w, h))
```

with output being:

```
Hello
Hello
Welcome Fred
width = 4 height = 5 area = 20
```

That example shows some more stuff that you can do with functions. Notice that you can use no arguments or two or more. Notice also when a function doesn't need to send back a value, a return is optional.

8.0.2 Variables in functions

When eliminating repeated code, you often have variables in the repeated code. In Python, these are dealt with in a special way. So far all variables we have seen are global variables. Functions have a special type of variable called local variables. These variables only exist while the function is running. When a local variable has the same name as another variable (such as a global variable), the local variable hides the other. Sound confusing? Well, these next examples (which are a bit contrived) should help clear things up.

```
a = 4

def print_func():
    a = 17
    print("in print_func a =", a)

print_func()
print("a = ", a)
```


When run, we will receive an output of:

```
in print_func a = 17
a = 4
```

Variable assignments inside a function do not override global variables, they exist only inside the function. Even though `a` was assigned a new value inside the function, this newly assigned value was only relevant to `print_func`, when the function finishes running, and the `a`'s values is printed again, we see the originally assigned values.

Here is another more complex example.

```
a_var = 10
b_var = 15
e_var = 25

def a_func(a_var):
    print("in a_func a_var =", a_var)
    b_var = 100 + a_var
    d_var = 2 * a_var
    print("in a_func b_var =", b_var)
    print("in a_func d_var =", d_var)
    print("in a_func e_var =", e_var)
    return b_var + 10

c_var = a_func(b_var)

print("a_var =", a_var)
print("b_var =", b_var)
print("c_var =", c_var)
print("d_var =", d_var)
```

output:

```
in a_func a_var = 15
in a_func b_var = 115
in a_func d_var = 30
in a_func e_var = 25
a_var = 10
b_var = 15
c_var = 125
d_var =

Traceback (most recent call last):
  File "C:\def2.py", line 19, in <module>
    print("d_var =", d_var)
NameError: name 'd_var' is not defined
```

In this example the variables `a_var`, `b_var`, and `d_var` are all local variables when they are inside the function `a_func`. After the statement `return b_var + 10` is run, they all cease to exist. The variable `a_var` is automatically a local variable since it is a parameter name. The variables `b_var` and `d_var` are local variables since they appear on the left of an equals sign in the function in the statements `b_var = 100 + a_var` and `d_var = 2 * a_var`.

Inside of the function `a_var` has no value assigned to it. When the function is called with `c_var = a_func(b_var)`, 15 is assigned to `a_var` since at that point in time `b_var` is 15,

making the call to the function `a_func(15)`. This ends up setting `a_var` to 15 when it is inside of `a_func`.

As you can see, once the function finishes running, the local variables `a_var` and `b_var` that had hidden the global variables of the same name are gone. Then the statement `print("a_var = ", a_var)` prints the value 10 rather than the value 15 since the local variable that hid the global variable is gone.

Another thing to notice is the `NameError` that happens at the end. This appears since the variable `d_var` no longer exists since `a_func` finished. All the local variables are deleted when the function exits. If you want to get something from a function, then you will have to use `return something`.

One last thing to notice is that the value of `e_var` remains unchanged inside `a_func` since it is not a parameter and it never appears on the left of an equals sign inside of the function `a_func`. When a global variable is accessed inside a function it is the global variable from the outside.

Functions allow local variables that exist only inside the function and can hide other variables that are outside the function.

8.0.3 Examples

temperature2.py

```
#!/usr/bin/python
#-*-coding: utf-8 -*-
# converts temperature to Fahrenheit or Celsius

def print_options():
    print("Options:")
    print(" 'p' print options")
    print(" 'c' convert from Celsius")
    print(" 'f' convert from Fahrenheit")
    print(" 'q' quit the program")

def celsius_to_fahrenheit(c_temp):
    return 9.0 / 5.0 * c_temp + 32

def fahrenheit_to_celsius(f_temp):
    return (f_temp - 32.0) * 5.0 / 9.0

choice = "p"
while choice != "q":
    if choice == "c":
        c_temp = float(input("Celsius temperature: "))
        print("Fahrenheit:", celsius_to_fahrenheit(c_temp))
        choice = input("option: ")
    elif choice == "f":
        f_temp = float(input("Fahrenheit temperature: "))
        print("Celsius:", fahrenheit_to_celsius(f_temp))
        choice = input("option: ")
    else:
        choice = "p"      #Alternatively choice != "q": so that print
                          #when anything unexpected inputed
    print_options()
    choice = input("option: ")
```